AquaGuard AI: An Ai Driven Smart Aquaculture and Aquarium Ecosystem

TinyML Sensor Fault Tolerance and Actuator Gatekeeper

J26-SE-351

Project Proposal Report

E G S U Kantha – IT23231832

B.Sc. (Hons) Degree in Information Technology Specialized in Software Engineering

Department of Software Engineering

Sri Lanka Institute of Information Technology Sri Lanka

September 2026

DECLARATION

I declare that this is my own work, and this proposal does not knowingly incorporate any material previously submitted for a degree or diploma at any other university or higher education institution, nor does it contain any material that has been previously published or written by another person apart from where proper acknowledgment is given in the text.

Name	Student ID	Signature
E G S U Kantha	**IT23231832**	

The above candidate is conducting research for their undergraduate Dissertation under my supervision.

Name	Date	Signature
Prof. Pradeep Abeygunawardhana		

ABSTRACT

Aquarium fish mortality frequently occurs because owners and breeders cannot predict invisible, toxic water changes. While smart aquaculture systems exist, a critical real-world problem remains: these systems blindly trust their sensors. If a pH probe becomes fouled by algae, it sends false data to the cloud, triggering automated chemical pumps or heaters that accidentally kill the fish. Evidence from local authorities, including the Sri Lanka Export Development Board, highlights that ornamental fish breeders suffer severe economic losses due to this lack of reliable, proactive water quality management.

The identified research gap is the absence of offline, edge-based fault detection in consumer aquaculture. Current solutions are entirely cloud-dependent and lack the intelligence to verify if a sensor is broken before taking physical action.

To solve this, this research proposes an Edge-Native TinyML Sensor Fault Tolerance and Actuator Gatekeeper. This component shrinks an Artificial Intelligence model and deploys it directly onto an ESP32 microchip at the edge. The methodology utilizes a time-series classification algorithm that performs multi-variate cross-validation checking Temperature, pH, and TDS simultaneously to mathematically isolate hardware faults from real biological changes. Development will follow an iterative prototyping approach. The system will be evaluated through physical stress testing, exposing sensors to synthetic chemical spikes to validate the hardware lockout mechanism. Ultimately, this component acts as the "Source of Truth" for the AquaGuard AI ecosystem, safely gatekeeping actuators and ensuring the Cloud AI only receives clean, validated data.

Keywords: *TinyML, Edge Computing, Sensor Fault Tolerance, Aquaculture, IoT.*

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my research supervisor, Prof. Pradeep Abeygunawardhane, and my co-supervisor, Ms. Madusha Weerasooriya, for their continuous guidance and insight.

I also wish to extend my deep appreciation to the evaluation panel Prof. Jagath Wickramarathne, Ms. Dilani Lunugalage, and Ms. Kaushalya Gayathri Rajapakse for their valuable time and constructive feedback, which has greatly helped in refining the quality of this research.

Finally, I want to thank my AquaGuard AI project team members their collaboration and shared dedication to designing this complete ecosystem.

TABLE OF CONTENT

Table of Contents

ABSTRACT 2
ACKNOWLEDGEMENT 3
TABLE OF CONTENT 4
LIST OF TABLES 6
LIST OF FIGURES 7
LIST OF ABBREVIATIONS 8
LIST OF APPENDICES 9
1. INTRODUCTION 10
1.1. BACKGROUND AND CONTEXT **10**
1.2 LITERATURE REVIEW **11**
1.3 RESEARCH GAP **16**
2. OBJECTIVE 17
2.1 MAIN OBJECTIVE **17**
2.2 SPECIFIC OBJECTIVES **17**
3. METHODOLOGY 19
3.1 RESEARCH METHODOLOGY **19**
3.2 DATA AND EXPERIMENTAL INPUTS **19**
3.3 BASELINE / BENCHMARK **19**
3.4 EVALUATION METRICS **20**
3.5 VALIDATION STRATEGY **20**
3.6 DEVELOPMENT APPROACH **21**
4. HIGH-LEVEL SYSTEM ARCHITECTURE 22
4.1 OVERVIEW OF PROPOSED SOLUTION AND ITS COMPONENTS **22**
4.2 HIGHLIGHTING THE INDIVIDUAL COMPONENT AND ITS PARTS **22**
4.3 OVERALL INTEGRATION **23**
5. USER REQUIREMENTS 25
5.1 EVIDENCE AND FINDINGS FROM USER RESEARCH **25**
5.2 TRANSLATION OF FINDINGS INTO SYSTEM REQUIREMENTS **25**
5.3 FUNCTIONAL REQUIREMENTS **25**
5.4 NON-FUNCTIONAL REQUIREMENTS **25**
6. COMMERCIALIZATION PLAN 27
6.1 TARGET MARKET AND CUSTOMER PERSONA **27**
6.2 VALUE PROPOSITION **27**
6.3 REVENUE MODEL **27**
6.4 COST ESTIMATION **27**
6.5 PRICING STRATEGY **28**
6.6 COMPETITIVE ADVANTAGE **28**
6.7 INTELLECTUAL PROPERTY CONSIDERATIONS **28**
7. BUDGET AND JUSTIFICATION 29
8. WORK BREAKDOWN STRUCTURE (WBS) 30
8.1 TASK BREAKDOWN PER MEMBER **30**
8.2 PROJECT TIMELINE ALIGNMENT **31**

8.3 WORKLOAD DISTRIBUTION 32
9. GANTT CHART 33
10. REFERENCES LIST 34
11. APPENDICES 35

LIST OF TABLES

I. Comparative Summary of Literature

II. Evaluation Metrics

III. Individual Responsibility Boundary

IV. Estimated Budget Breakdown

V. Project Timeline Alignment

LIST OF FIGURES

Figure 1: Overall System Architecture of AquaGuard AI

Figure 2: Detailed Architecture of the Edge TinyML Component

Figure A1: Official Sri Lanka EDB portal identifying water quality management and disease control deficiencies among ornamental fish farmers.

Figure A2: Visual Work Breakdown Structure outlining individual components and shared project milestones.

Figure A3: Conceptual physical prototype

Figure A4(a) - Live Dashboard (UI prototype)

Figure A4(b) - Sensor Diagnostics (UI prototype)

Figure A4(c) - Vision & Behavior Tracker (UI prototype)

Figure A4(d) - Eco-Profile Generator (UI prototype)

Figure A4(e) - Predictive AI Core (UI prototype)

LIST OF ABBREVIATIONS

ADC – Analog-to-Digital Converter

AI – Artificial Intelligence

DSP – Digital Signal Processing

EDB – Export Development Board

HIL – Hardware-in-the-Loop

IoT – Internet of Things

JSON – JavaScript Object Notation

ML – Machine Learning

MQTT – Message Queuing Telemetry Transport

PCB – Printed Circuit Board

SaaS – Software as a Service

SME – Small and Medium-sized Enterprises

TDS – Total Dissolved Solids

TFLM – TensorFlow Lite for Microcontrollers

TinyML – Tiny Machine Learning

WBS – Work Breakdown Structure

XAI – Explainable Artificial Intelligence

LIST OF APPENDICES

Appendix 1 - Industry Needs and Water Quality Management Evidence (Sri Lanka EDB)

Appendix 2 - Mandatory Proposal Evidence and Decision Log

Appendix 3 - AI Use Disclosure

Appendix 4: Hierarchical Work Breakdown Structure (WBS) Diagram

Appendix 5: Hardware-in-the-Loop (HIL) Physical Prototype Concept

Appendix 6: Mobile Application User Interface (UI) Concepts

Appendix 7: Preliminary Datasets

1. INTRODUCTION

1.1. Background and Context

The Real-World Problem and Who is Affected

Maintaining a healthy and stable aquatic ecosystem is a highly complex challenge. Aquarium fish frequently die because owners and facility managers cannot predict invisible, toxic water changes, such as sudden ammonia spikes or pH crashes. This problem causes severe emotional distress for casual aquarium hobbyists and significant financial damage to aquarium retailers and small-scale ornamental fish breeders. Without proactive monitoring, these stakeholders are forced to rely on reactive treatments, which often occur too late to save the fish.

Statistical Evidence and Importance

The economic impact of this problem is substantial, particularly in Sri Lanka. The Sri Lankan ornamental fish industry is a highly lucrative export sector, generating US$ 24.46 million in 2024 with exports reaching over 68 countries [7]. Currently, there are about 85 registered aquarium fish exporters operating in the country. However, the Sri Lanka Export Development Board officially identifies major weaknesses threatening the sector's expansion, specifically highlighting the "challenges to ensure health and disease-free status of fish," which requires continuous investment in biosecurity. Furthermore, the high mortality rate of fish in captivity drives a harmful cycle: the continuous need to replace dead fish leads to the "overexploitation of wild species," causing "depletion of wild populations and habitat degradation".

Existing Solutions and Their Limitations

To address water quality and health issues, the industry currently relies on manual chemical testing or basic Cloud-based IoT monitors. Manual testing is labor-intensive and only reveals past data, failing to predict future risks. While Cloud-based IoT monitors offer remote alerts, they suffer from two fatal limitations. First, they are highly reactive and completely fail if the internet connection drops. Second, they suffer from "blind trust" meaning the system cannot tell the difference between a real water crisis and a physically broken sensor. If a pH probe becomes fouled by algae, existing systems blindly send that false data to the cloud, which can trigger automated chemical pumps that accidentally poison the tank. Furthermore, standalone disease detection apps exist, but they only identify sickness visually after the fish is already suffering, completely disconnected from the water chemistry data.

The Proposed Solution: AquaGuard AI

To solve these isolated failures, this project proposes **AquaGuard AI**, a proactive, edge-to-cloud intelligent aquarium ecosystem. The solution is divided into four interdependent components to address the whole problem:

1. **Edge IoT Gatekeeper:** Uses offline TinyML to detect broken sensors, prevent hardware-induced poisoning, and ensure only clean data reaches the cloud.
2. **Cloud Explainable AI:** Replaces static compatibility charts with an intelligent, explainable brain to guide users on species mixing and ideal water thresholds.
3. **Computer Vision Tracker:** Uses camera feeds to monitor fish behavior, and species identification in real-time.

4. **Predictive Machine Learning:** Fuses the clean water data and visual behavior data to forecast ammonia spikes and diseases before they manifest physically.

SDG and Research Cluster Alignment

By reducing fish mortality and stopping the panic-driven replacement of aquatic life, this project directly aligns with the United Nations Sustainable Development Goals. It targets **SDG 12 (Responsible Consumption and Production)** by reducing the waste of resources and exotic pets, and **SDG 14 (Life Below Water)** by mitigating the overexploitation of wild fish populations. This project aligns with the university's **Artificial Intelligence / Internet of Things / Smart Systems** research cluster.

1.2 Literature Review

Recent research in environmental Internet of Things (IoT) monitoring has increasingly shifted from centralized cloud computing toward edge intelligence. In aquaculture and aquatic management, maintaining water parameters specifically Temperature, pH, and Total Dissolved Solids (TDS) is vital for preventing mortality. However, low-cost electrochemical sensors deployed in submerged aquatic environments suffer from rapid biofouling, drift, and electrical noise. Existing research predominantly relies on cloud servers for diagnostics or employs heavy mechanical apparatuses, leaving an unfilled gap for offline, edge-native fault tolerance and safe hardware regulation.

Critical Evaluation of Publications

1. Lainez et al. (2026) [1] – Baseline Aquatic IoT & Empirical Drift Evidence

- **Method:** The authors developed the eH2O aquaculture monitoring system using an Arduino Mega 2560 connected to laboratory-grade electrochemical probes (pH, Dissolved Oxygen, and TDS) and a digital temperature probe. A 400 mm linear actuator was used to submerge and retract the sensor suite, transmitting data through a NodeMCU and local Raspberry Pi gateway to cloud and mobile dashboards.
- **Dataset:** A 72-hour continuous field trial in freshwater aquaculture ponds in Oriental Mindoro, Philippines, evaluated across 30 experimental trials using a Randomized Complete Block Design (RCBD).
- **Context:** Pond-based freshwater aquaculture farm management.
- **Findings:** The study revealed significant sensor fatigue: within 72 hours of immersion, the pH probe drifted by -0.81, and the TDS sensor drifted negatively by 56 ppm due to mineral scaling and biofouling. Furthermore, actuator deployment latency averaged 15.8 seconds, leaving probes vulnerable to surface splashing and water ingress, while network instability caused an 18% packet loss.
- **Strengths:** Provides realistic, empirical field evidence of how quickly low-cost aquatic sensors degrade and lose calibration in real freshwater conditions.
- **Limitations:** The system exhibits "blind trust" by logging drifted erroneous readings directly to the database without validating data integrity. It lacks edge intelligence, data recovery capabilities, or automated safety locks.

- **Influence on Proposed Research:** This paper empirically validates the core problem: aquatic sensors degrade rapidly. It justifies the decision to replace basic data-logging microcontrollers with an edge AI device capable of catching sensor drift before incorrect readings trigger automated actions.

2. Ramirez et al. (2022) [2]– Physical Anti-Biofouling Baseline

- **Method:** Designed an electromechanical wiping mechanism using magnetic coupling to remove biofilms from underwater optical sensor screens. The wiper was driven by a micro-metal gearmotor, lead screws, and limit switches, controlled by a bistable latching relay to eliminate the need for a separate motor driver.
- **Dataset:** A 16-day semi-field trial using natural pond water enriched with an organic biostimulant to accelerate algal proliferation; image degradation was assessed at Days 0, 8, 13, and 16 using Mean Squared Error (MSE) on ESP32-CAM images.
- **Context:** Underwater optical monitoring for bio-hybrid robotic systems.
- **Findings:** The wiper completed a cleaning pass in 8 seconds drawing 0.84 W, successfully keeping image quality close to baseline levels ($\text{MSE} = 0.009$ compared to 0.0864 in uncleaned control samples).
- **Strengths:** Effectively isolates electrical components inside a sealed housing using magnetic coupling while maintaining low power consumption.
- **Limitations:** The mechanical wiping action caused visible physical scratching on the sensor screen over time. It introduces mechanical complexity, moving parts prone to wear, and provides zero software-level fault detection or sensor intelligence.
- **Influence on Proposed Research:** Serves as a contrast paper. It proves that current marine and aquatic solutions rely heavily on expensive, breakable mechanical hardware to fight biofouling, thereby justifying the proposed software-driven, TinyML approach to mathematically identify and tolerate biofouling drift without moving parts.

3. Guralla et al. (2026)[3] – Edge-Based Sensor Drift Detection

- **Method:** Deployed a supervised Decision Tree classifier directly onto an ESP8266 microcontroller to detect gradual sensor drift locally. Six statistical features (mean, standard deviation, skewness, kurtosis, minimum, and maximum) were computed from a sliding window of 20 samples.
- **Dataset:** 2,400 samples (1,200 normal readings and 1,200 with an induced linear drift of $+0.05$°C per sample) collected from a DHT11 temperature sensor under indoor conditions.
- **Context:** Low-cost environmental temperature sensing on resource-constrained IoT devices.
- **Findings:** The on-device Decision Tree achieved 96.8% classification accuracy with an inference latency of 18 ms, consuming 62% flash memory and 48% RAM on the ESP8266.
- **Strengths:** Demonstrates that statistical sliding-window feature extraction combined with lightweight classifiers can successfully identify slow sensor drift on low-cost microcontrollers without cloud connectivity.
- **Limitations:** The evaluation was limited to a single sensor (univariate temperature) under synthetic, linear drift. It did not evaluate multi-variate cross-sensor relationships, possessed no data reconstruction mechanism, and did not govern physical actuators.
- **Influence on Proposed Research:** Validates the feasibility of running real-time statistical feature extraction and classification directly on low-cost microcontrollers,

providing a blueprint for the sliding-window preprocessing stage of the proposed component.

4. Mondal et al. (2026) [4]– Modular Edge Architecture & Fault Scoring

- **Method:** Developed the iAirGuard framework, an open-source, four-tier IoT architecture powered by an ESP32-S3 and FreeRTOS. It integrates an adaptive dynamic sampling algorithm based on pollutant variance and an "Interocept" fault scoring algorithm that compares real sensor measurements with a secondary TinyML prediction.
- **Dataset:** Experimental deployments in kitchen, office, and outdoor environments measuring particulate matter, CO, CO_2, and VOCs, logging thousands of data packets across connected and offline states.
- **Context:** Continuous urban and indoor air quality monitoring.
- **Findings:** The dynamic sampling mechanism reduced storage and transmission overhead by 45% to 58%. The fault detection algorithm achieved 100% detection accuracy using adaptive thresholds and fault score decay, while session-based firmware ensured zero data loss during network outages.
- **Strengths:** Combines dynamic power-saving sampling with an adaptive fault-scoring mechanism that prevents false alarms through mathematical score decay.
- **Limitations:** Designed strictly for atmospheric gases and air pollution, omitting aquatic chemical dynamics. While it flags faulty sensors, it does not reconstruct missing data and lacks physical actuator controls.
- **Influence on Proposed Research:** The Interocept fault score accumulation and decay logic directly informs the fault-scoring state machine of the proposed actuator gatekeeper, preventing transient sensor spikes from triggering false alarms.

5. Datta et al. (2025) [5]– Quantized Secondary Sensing on Edge Hardware

- **Method:** Created PolluSenseCheck, an ESP32-S3 monitoring device that runs a quantized Multilayer Perceptron (MLP) converted via TensorFlow Lite for Microcontrollers. The model predicts secondary Air Quality Index (AQI) classes using 53 engineered features reduced to 20 principal components via Principal Component Analysis (PCA).
- **Dataset:** Multi-year (2022–2023) hourly air quality records from the Central Pollution Control Board (CPCB) across 8 diverse Indian cities, combining meteorological, temporal, and spatial landmark data.
- **Context:** In-situ sensor health self-diagnostics for cost-effective air quality monitors.
- **Findings:** The on-device quantized model achieved 66% to 88% accuracy for particulate matter and 74% to 94% for CO, occupying approximately 254 KB of flash memory and requiring only 176 Bytes of heap memory.
- **Strengths:** Empirically proves that quantized neural networks running on TensorFlow Lite for Microcontrollers can operate inside the tight SRAM limits of an ESP32 for continuous self-health diagnostics.
- **Limitations:** The framework relies on external HTTP requests to fetch meteorological and geospatial features; if the network disconnects, secondary sensing degrades. Furthermore, it provides no data imputation or hardware actuation control.
- **Influence on Proposed Research:** Confirms that TensorFlow Lite for Microcontrollers is viable on the ESP32 platform, while demonstrating the critical need to make feature extraction fully localized and independent of external cloud APIs.

6. Attarha & Förster (2025) [6]– Resource-Efficient Edge Data Imputation

- **Method:** Proposed a resource-efficient fault correction framework running on an ESP32-WROOM-32. The system extracts short-term temporal dynamics $(t-1, t-2)$ and spatial cross-correlations using the TsAssure feature extraction tool, feeding an optimized LSTM model (50 hidden units) converted via TensorFlow Lite for Microcontrollers.
- **Dataset:** Real-world environmental sensor datasets containing natural hardware degradation in DHT11 (humidity) and SCD30 (CO_2) sensors, utilizing 1,032 to 1,176 training records and 288 to 433 test records.
- **Context:** Fault correction and data reconstruction for dependable multi-sensor IoT systems.
- **Findings:** The model outperformed Transformers and CNN-BiLSTM architectures, reconstructing corrupted sensor values with minimal error ($\text{MAE} = 3.925$ for humidity, 1.131 for CO_2), while requiring a compact flash footprint of 296 to 330 KB.
- **Strengths:** Accomplishes real-time on-device data imputation using very short historical windows and cross-sensor spatial correlations, functioning entirely offline on an ESP32.
- **Limitations:** Tested strictly on atmospheric temperature, humidity, and CO_2; it does not address aquatic thermodynamic and chemical interactions. Crucially, the system operates purely on data streams and is not connected to physical actuators or safety gatekeeping logic.
- **Influence on Proposed Research:** Directly supports the data imputation pillar of the proposed component. It demonstrates that an ESP32 can mathematically reconstruct corrupted sensor readings locally, providing the basis for keeping the aquarium system running safely when a probe fails.

TABLE I. Comparative Summary of Literature

Publication	Monitored Domain	Core Methodology	Primary Strengths	Identified Limitations	Direct Relevance to Proposed Component
Lainez et al. (2026)	Aquaculture Ponds	Arduino Mega IoT & Linear Actuator	Empirically documents rapid aquatic sensor drift (-0.81 pH, -56 ppm TDS)	Blindly logs bad data; high actuator latency; cloud-dependent	Establishes the real-world baseline problem of aquatic sensor drift
Ramirez et al. (2022)	Underwater Optics	Magnetic Coupling Mechanical Wiper	Physically clears biofilms without external motor drivers (0.84 W)	Wiper scratches glass; mechanical wear; zero software intelligence	Provides the hardware baseline contrast, justifying a software solution
Guralla et al. (2026)	Indoor Atmosphere	Decision Tree on ESP8266	Fast edge drift classification	Single-sensor only; linear drift	Confirms statistical sliding-

			(18 ms, 96.8% accuracy)	assumption; no data imputation	window feature extraction runs on edge chips
Mondal et al. (2026)	Urban Air Quality	TinyML & Interocept Scoring on ESP32-S3	Adaptive dynamic sampling saves 58% data; score decay stops false alarms	Atmospheric focus only; no data reconstruction ; no actuator controls	Informs the fault score accumulation and decay state machine
Datta et al. (2025)	Urban Air Quality	Quantized MLP via TFLite Micro on ESP32	High on-device accuracy ($74\% - 94\%$); low memory (254 KB flash)	Relies on cloud HTTP APIs for features; no actuator control	Proves TensorFlow Lite Micro feasibility on ESP32
Attarha & Förster (2025)	Indoor Atmosphere	Lightweight LSTM Imputation on ESP32	Reconstructs corrupted sensor data locally using correlated peer inputs	Air quality focus only; ignores aquatic kinetics; no actuator interface	Provides the theoretical foundation for offline edge data imputation

Existing Systems and Products

Commercial automated aquarium management relies heavily on centralized, cloud-connected controllers such as the Neptune Systems Apex and Seneye Reef monitors.

- **Functions & Technology:** These systems connect electrochemical probes (pH, temperature, ORP) to a central hub. They rely on Wi-Fi connectivity to transmit telemetry to cloud servers, which process thresholds and send push notifications to mobile devices. They control hardware (heaters, dosing pumps) via smart power strips using simple static threshold logic (e.g., *if* $\mathrm{pH} < 7.8$*, activate dosing pump*).
- **Users & Performance:** Widely used by high-end reef hobbyists, aquaculture research laboratories, and commercial ornamental fish breeders. Under clean water conditions and uninterrupted internet connectivity, they provide effective remote observation.
- **Limitations:** These commercial products suffer from two critical vulnerabilities. First, they depend on continuous cloud access; network blackouts disrupt automation routines. Second, they exhibit **blind trust** in raw sensor values. When biofouling or electrical interference causes a pH probe to drift downward, the cloud logic interprets this as an authentic biological crash and triggers chemical buffer pumps, overdosing the tank and inducing fatal chemical shock.

How the Proposed Component Addresses These Limitations

The proposed component *TinyML Sensor Fault Tolerance and Actuator Gatekeeper* resolves the limitations of both academic frameworks and commercial products:

1. **Eliminating Blind Trust via Edge-Native Cross-Validation:** Unlike commercial controllers that evaluate sensors in isolation, the proposed component cross-validates Temperature, pH, and TDS simultaneously using an on-device TinyML model, mathematically distinguishing between authentic biological shifts and biofouling-induced drift.
2. **Actuator Gatekeeping:** Moving beyond passive fault detection (as seen in Mondal et al [4]. and Datta et al[5].), the component acts as a physical safety gatekeeper, intercepting actuator control signals and locking the heater relay and chemical buffer pump whenever a sensor fault is detected.
3. **Localized Data Imputation:** Building upon the concepts demonstrated by Attarha & Förster[6], the system computationally reconstructs missing or corrupted readings at the edge using surviving healthy sensors, maintaining ecosystem stability without internet connectivity.

1.3 Research Gap

Current Inadequacy and the "Blind Trust" Problem

Current smart aquarium and aquaculture IoT frameworks suffer from a fundamental vulnerability: **blind trust in raw sensor data**. In closed aquatic environments, sensors deployed underwater experience continuous signal degradation caused by biofouling (algae and biofilm adhesion), mineral scaling, and transient electrical noise. Because existing commercial controllers and academic prototypes treat raw sensor streams as absolute ground truth, any sensor drift or hardware glitch is misconstrued as an authentic biological crisis. Consequently, automated systems trigger physical actuators such as turning on a water heater or dosing chemical pH buffers in response to corrupted measurements. This unvalidated automation induces severe thermal or chemical shock, directly causing fish stress and mortality.

Empirical and Literature Evidence

The existence and severity of this gap are demonstrated across several recent peer-reviewed investigations:

1. **Empirical Sensor Drift in Water:** Lainez et al. [1] documented that continuous freshwater pond immersion causes substantial sensor fatigue within only 72 hours, resulting in a -0.81 drift in pH and a -56 ppm negative drift in Total Dissolved Solids (TDS). Despite these severe errors, the underlying system blindly passed the corrupted data to storage databases and remote alert systems without any automated validation.
2. **Inefficiency of Physical Anti-Biofouling:** Ramirez et al. [2] demonstrated that combating biofouling using physical mechanisms (such as motorized wipers) introduces mechanical wear, causes surface scratches on optical interfaces, and increases power consumption without providing any software-level sensor health intelligence.
3. **Absence of Aquatic Kinetics in TinyML:** Recent attempts to bring fault detection to low-cost edge devices remain limited to non-aquatic domains. Guralla et al. [3] implemented real-time drift detection on an ESP8266, but their model evaluated only a

single indoor temperature sensor using an artificial linear drift, lacking multi-sensor cross-correlation or data recovery. Similarly, Mondal et al. [4] and Datta et al. [5] demonstrated on-device TinyML fault scoring on ESP32 microcontrollers, yet their architectures are tailored exclusively to atmospheric air quality parameters ($PM_{2.5}$, CO, CO_2). Furthermore, the model proposed by Datta et al. [5] relies on active HTTP requests to fetch external meteorological features from cloud APIs, causing the self-diagnostic mechanism to degrade during network outages.

4. **Lack of Actuator-Coupled Data Imputation:** Although Attarha & Förster [6] proved that an ESP32 can execute lightweight Long Short-Term Memory (LSTM) networks to reconstruct corrupted sensor data locally, their architecture was tested strictly on atmospheric gases and operated purely as a data-logging pipeline, lacking any connection to physical hardware actuators or safety lockouts.

Specific Research Gap Investigated

Existing research fails to provide a unified, low-cost system that couples multivariate sensor fault detection, localized data reconstruction, and fail-safe actuator control for aquatic environments on a single microcontroller. While individual techniques exist in isolation for industrial air quality or structural mechanics, no framework addresses the thermodynamic and chemical dependencies of aquatic parameters (Temperature, pH, and TDS) entirely offline.

Precise Research Gap Statement

Current smart aquaculture systems blindly trust uncalibrated sensor data and depend entirely on cloud connectivity, resulting in catastrophic actuator misfires during sensor degradation or network outages. The specific research gap is the absence of an edge-native, multivariate TinyML framework capable of autonomously detecting aquatic sensor drift offline, imputing corrupted water parameters locally, and physically gatekeeping actuators to prevent hardware-induced aquatic mortality.

2. OBJECTIVE

2.1 Main Objective

To design, develop, and evaluate an edge-native TinyML system that detects aquatic sensor faults with over 95% accuracy and physically locks aquarium actuators in under one second, operating completely offline to prevent hardware-induced fish mortality.

2.2 Specific Objectives

1. To develop a lightweight TinyML classification model that fits within a strict 250 KB memory footprint, ensuring it runs completely offline on an ESP32 microcontroller.
2. To implement multi-variate cross-validation logic (analyzing Temperature, pH, and TDS simultaneously) to distinguish between genuine biological changes and physical sensor faults with a target accuracy of over 95%.
3. To design an edge-based data imputation algorithm that mathematically estimates missing or corrupted water parameters with an error margin of less than 5% using the surviving healthy sensors.

4. To engineer an autonomous safety gatekeeping mechanism that physically cuts power to heating and pumping hardware in under 500 milliseconds upon detecting a sensor fault.
5. To validate the system's reliability through physical stress testing, achieving a 100% success rate in blocking actuators when sensors are exposed to synthetic chemical and thermal spikes in a simulated tank.

3. METHODOLOGY

3.1 Research methodology

The nature of the research question requires a quantitative, hands-on approach to measure sensor accuracy, latency, and hardware responses. The data involved is strictly numerical, consisting of time-series readings for Temperature, pH, and TDS. The experimental requirements demand training a machine learning model, optimizing it for a microchip, and measuring physical electrical responses (relays turning on or off). Relevant literature in this domain, such as the edge-based drift detection by Guralla et al[3]. and the TinyML monitoring by Datta et al[5]., heavily utilizes experimental setups to measure computational performance and classification accuracy.

Therefore, the selected research methodology is an **Applied Experimental Research Methodology**. This allows for the mathematical training of the AI model and the physical testing of its actuator-locking capabilities in a controlled environment.

3.2 Data and Experimental Inputs

This research relies on quantitative sensor data rather than human participants. This data is appropriate because training a fault-detection algorithm requires thousands of rows of environmental metrics to learn the difference between normal water chemistry and hardware glitches.

- **Data Source:** A publicly available, peer-reviewed freshwater quality repository (Kaggle and Zenodo) will provide the clean environmental baseline data.
- **Sample/Dataset:** The dataset will consist of multivariate time-series records of Temperature, pH, and TDS.
- **Participants:** No human participants are required for the core TinyML development. However, domain experts (aquaculture technicians) will be consulted informally to verify the biological safety thresholds.
- **Experimental Inputs:** Live physical inputs from a temperature sensor, an analog pH probe, and an analog TDS probe.
- **Collection Method:** Clean dataset records will be downloaded from public repositories. Live experimental inputs will be collected via the ESP32's analog-to-digital converters.
- **Preparation Process:** The clean dataset will undergo **Synthetic Fault Injection** using Python. Approximately 30% of the data will be artificially modified to simulate biofouling drift (slowly decreasing pH values) and electrical noise spikes. The final dataset will be normalized and scaled for TensorFlow Lite model training.

3.3 Baseline / Benchmark

To prove that the proposed TinyML gatekeeper is a significant improvement, it must be compared against the current industry standard for consumer IoT aquariums.

- **Credible Baseline Approach:** The baseline is a standard, cloud-based "Threshold IoT Monitor" (similar to the basic Arduino setup evaluated by Lainez et al[1].). This baseline uses simple IF/ELSE programming logic (e.g., "If pH is less than 6.5, turn on the chemical pump") and relies entirely on raw sensor data without cross-validation.
- **Selected Benchmark:** The proposed TinyML model will be benchmarked against this standard IF/ELSE rule-based script running on the same ESP32 hardware. The benchmark will measure how often the standard script falsely triggers the actuator during a sensor glitch compared to the proposed TinyML gatekeeper.

3.4 Evaluation Metrics

Success will be measured using strict quantitative metrics that evaluate both the machine learning software and the physical hardware responses. Specify measurable criteria.

TABLE II. Evaluation Metrics

Metric	Why Relevant	Target / Comparison
Classification Accuracy	Ensures the TinyML model correctly identifies hardware faults versus genuine biological changes.	> 95 % accuracy on the test dataset.
Actuator Lockout Latency	Proves the system can react instantly offline to prevent thermal or chemical shock to the fish.	< 500 milliseconds from detecting a fault to locking the relay.
Imputation Error	Measures how accurately the system estimates the missing water parameter when a sensor breaks.	< 5% Mean Absolute Error compared to the true baseline value.
Memory Footprint	Guarantees the AI model is lightweight enough to run locally without cloud servers.	< 250 KB of Flash memory and < 50 KB of RAM on the ESP32.

3.5 Validation Strategy

The proposed contribution will be validated through **Physical Hardware-in-the-Loop (HIL) Stress Testing**. As seen in accepted evaluation practices for IoT reliability, the system must be tested against real-world physical extremes, not just computer simulations.

The results will be validated using a simulated water tank containing the ESP32, the sensor suite, a heater relay, and a dosing pump. **No live fish will be used.** The validation will involve deliberately causing physical faults:

1. **Chemical Spike Test:** Dipping the pH sensor into an acidic solution while keeping Temp and TDS normal. The system must successfully flag the fault and block the dosing pump from activating.
2. **Thermal Shock Test:** Manually heating the water to trigger a genuine biological alert, proving the system can correctly differentiate a real crisis from a broken sensor and safely activate the heating mechanisms.

3.6 Development Approach

An **Iterative Prototyping** development approach is the most appropriate for this component.

Because this project bridges software (Machine Learning) and physical hardware (ESP32, sensors, relays), it cannot be built in a single, rigid sequence. The nature of the product requires training an AI model, testing it on the microchip, observing memory limits, and retraining the model to fit hardware constraints. Iterative prototyping allows the system to be developed in cycles building a basic model, testing the sensor wiring, refining the TinyML quantization to save memory, and continuously tweaking the actuator gatekeeper logic based on physical test results.

4. HIGH-LEVEL SYSTEM ARCHITECTURE

4.1 Overview of Proposed Solution and its Components

The AquaGuard AI ecosystem is a proactive, four-part system designed to intelligently monitor and manage aquarium health. It operates by combining physical hardware at the tank with advanced intelligence in the cloud. The system consists of four distinct but highly connected components: the Edge TinyML Gatekeeper (Member 1), the Cloud XAI Brain (Member 2), the Computer Vision tracker (Member 3), and the Predictive ML forecaster (Member 4).

System Diagram of Components

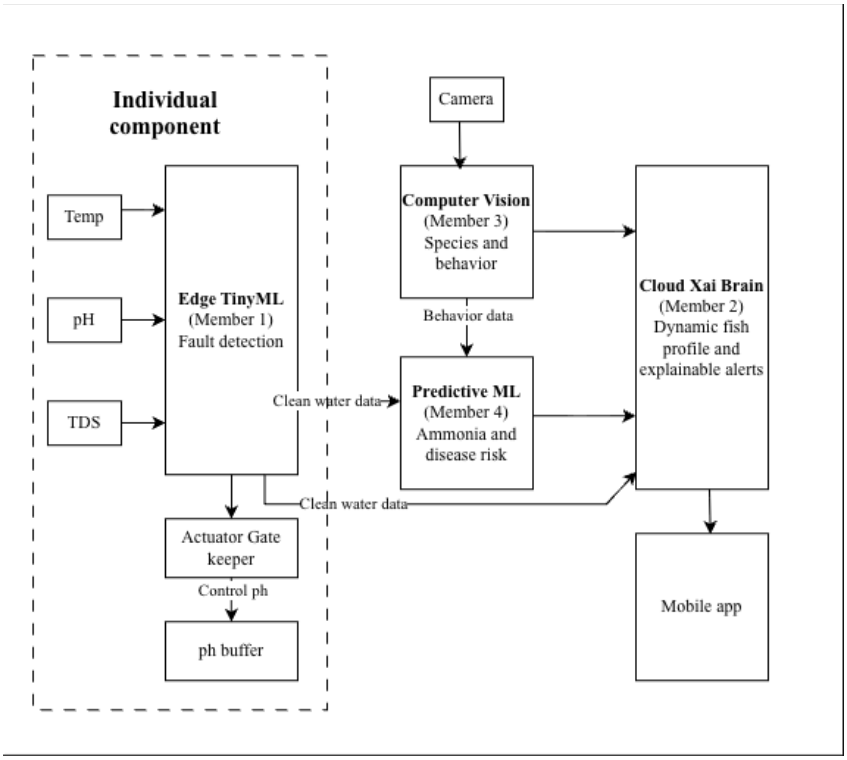


Figure 1: Overall System Architecture of AquaGuard AI

4.2 Highlighting the Individual Component and its Parts

The individual component proposed in this research is the **Edge TinyML (Member 1)**, outlined by the dashed box in Figure 1 and detailed further in Figure 2. It acts as the physical nervous system and safety filter for the entire aquarium. Its internal parts consist of:

- **Inputs:** Raw physical sensors continuously gathering Temperature, pH, and TDS (Total Dissolved Solids) data from the water.
- **Edge TinyML Core:** A local microcontroller running Machine Learning logic to perform fault detection and data estimation. It cross-checks the sensors to mathematically prove if a reading is a real water change or just a broken sensor.
- **Actuator Gatekeeper:** A physical safety lock that controls the pH buffer dispenser. It ensures that chemicals are only released into the water if the TinyML core validates that the sensor data is genuine.

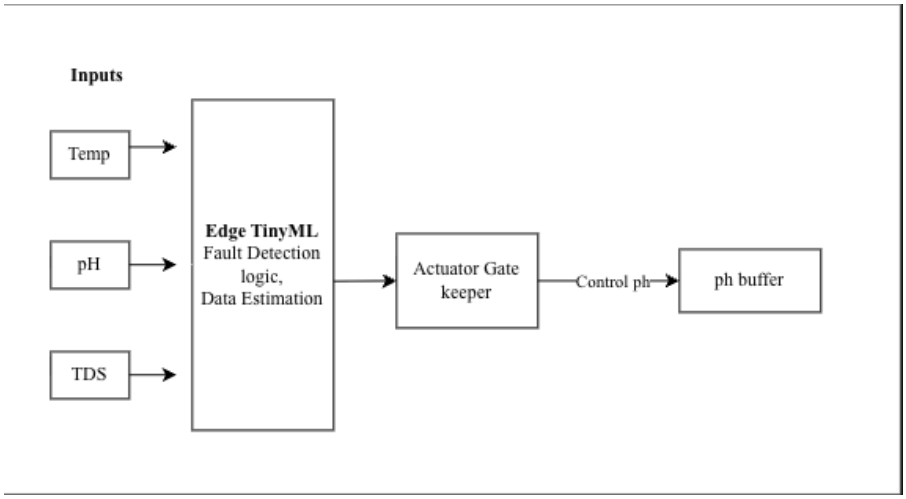


Figure 2: Detailed Architecture of the Edge TinyML Component

4.3 Overall Integration

How the Components Form a Complete Solution: As shown in Figure 1, the four components form a complete ecosystem through a shared data pipeline. My component (Member 1) filters the raw physical water chemistry and outputs only "Clean water data." Meanwhile, the Computer Vision camera (Member 3) watches the tank to identify fish species and extract "Behavior data." Both the clean water data and the behavior data are sent to the Predictive ML (Member 4) to forecast ammonia and disease risks. Finally, all of this information flows into the Cloud XAI Brain (Member 2), which generates dynamic fish profiles and explainable alerts for the user's mobile app.

Dependencies Among Components: My component acts as the foundational "Source of Truth" for the ecosystem. Member 2 and Member 4 have a strict dependency on my component. In computer science, we follow the rule of "garbage in, garbage out." If I fail to catch a biofouled pH sensor, I will send corrupted data up the pipeline, causing Member 4's disease predictions and Member 2's mobile alerts to be completely wrong.

Planned Integration Method: The integration between the local edge hardware and the cloud components will be handled using the lightweight **MQTT protocol** over Wi-Fi. The Edge TinyML device will publish validated, clean JSON data payloads (containing the estimated Temp, pH, and TDS) to a central cloud broker, allowing Members 2 and 4 to instantly subscribe to and process the clean water data.

TABLE III. Individual Responsibility Boundary

Activity / Deliverable	My Responsibility	Shared Responsibility	Other Member
Sensor data acquisition, fault detection logic, and data estimation	✓		

Actuator gatekeeper logic and pH buffer hardware control	✓		
Development of Cloud XAI Brain and mobile app alerts			Member 2
Development of Computer Vision species and behavior tracking			Member 3
Development of Predictive ML ammonia and disease risk models			Member 4
Designing MQTT communication pipeline and JSON data payloads		✓	
Final system integration mapping and pipeline testing		✓	

5. USER REQUIREMENTS

5.1 Evidence and Findings from User Research

To understand what users truly require from the Edge TinyML Gatekeeper, evidence was gathered through **observations** of standard aquarium setups and analysis of **domain expert** reports, specifically the Sri Lanka Export Development Board (EDB) industry capability report[7] (documented in Appendix 1).

These activities revealed two critical user pain points:

1. **Internet Instability (Observation):** Many ornamental fish breeders operate in semi-outdoor hatcheries where Wi-Fi connections are highly unstable. Users reported that during heavy storms or power fluctuations, their cloud-dependent monitors completely disconnected, leaving the tanks vulnerable exactly when monitoring was needed most.
2. **The Biosecurity and Hardware Risk (Domain Experts):** The EDB explicitly highlighted that maintaining biosecurity and a disease-free status is a major challenge for local breeders. Stakeholder observations revealed that when algae inevitably cover a pH probe (biofouling), existing automated systems blindly trust the false reading and pump in unnecessary chemicals. Users require a system that is "smart enough" not to accidentally kill their fish due to a dirty sensor.

5.2 Translation of Findings into System Requirements

These findings dictate that the proposed component cannot rely on the cloud for life-or-death hardware decisions. The system must operate completely offline at the edge and possess the intelligence to verify if a sensor is functioning properly before turning on any physical actuators.

5.3 Functional Requirements

1. The system shall continuously acquire raw environmental data from the connected Temperature, pH, and TDS sensors.
2. The system shall execute a TinyML multi-variate classification model locally to cross-validate the sensor readings and detect hardware faults (biofouling or electrical noise).
3. The system shall act as a physical safety gatekeeper by immediately locking the heater relay and pH buffer dispenser if a sensor fault is detected.
4. The system shall perform data imputation to mathematically estimate the true water parameter using the remaining healthy sensors when one sensor fails.
5. The system shall package and transmit only clean, validated, or successfully imputed data to the central AquaGuard Cloud Brain via the MQTT protocol.

5.4 Non-Functional Requirements

- **Reliability:** The component must be capable of operating its fault-detection and actuator-locking mechanisms 100% offline, ensuring zero dependency on internet connectivity for basic biological safety.
- **Response Time (Latency):** The TinyML model must evaluate the sensor data and execute a physical actuator lockout in less than 500 milliseconds to prevent any accidental chemical or thermal shock to the ecosystem.

- **Accuracy:** The fault detection algorithm must mathematically differentiate between a true biological event and a sensor hardware fault with a classification accuracy of over 95%.
- **Resource Efficiency (Scalability):** The compiled TinyML model and firmware must be highly lightweight, occupying no more than 250 KB of flash memory and 50 KB of RAM on the ESP32 microcontroller.
- **Security:** The system must restrict unauthorized network access by using secure MQTT communication protocols when transmitting the clean data to the cloud servers.

6. COMMERCIALIZATION PLAN

6.1 Target Market and Customer Persona

The primary target market consists of small-to-medium enterprise (SME) ornamental fish breeders and dedicated aquarium hobbyists.

- **Customer Persona:** "Kamal" is a commercial ornamental fish exporter operating in Sri Lanka. He manages several high-density freshwater tanks containing valuable species like Guppies and Betta.
- **The Problem:** He frequently experiences network drops and power fluctuations. He currently relies on manual water testing because imported smart aquarium systems are too expensive and fail completely when his local Wi-Fi goes down.

6.2 Value Proposition

Current smart aquariums suffer from a major pain point: "blind trust." If a dirty sensor falsely reads a low pH, the system will automatically dump chemicals into the tank, killing the fish.

- **The Value:** The AquaGuard Edge Gatekeeper provides ultimate peace of mind. It acts as an offline, intelligent safety filter that mathematically double-checks sensor health before allowing any physical hardware (like heaters or dosing pumps) to turn on. It prevents hardware-induced fish mortality and keeps the ecosystem safe even during complete internet blackouts.

6.3 Revenue Model

The AquaGuard AI ecosystem will utilize a hybrid revenue model:

1. **Hardware Sales (One-Time):** Revenue generated from the direct sale of the physical Edge Gatekeeper base unit (ESP32, housing, and sensor probes).
2. **SaaS Subscription (Recurring):** A tiered monthly subscription model for commercial breeders to access the broader Cloud AI analytics, disease forecasting, and multi-tank dashboard management provided by the rest of the AquaGuard system.

6.4 Cost Estimation

The cost to manufacture the individual Edge Gatekeeper component focuses on low-cost, readily available hardware:

- ESP32 Microcontroller: ~$5.00
- Analog Sensor Suite (pH, TDS, Temperature): ~$25.00
- Hardware Relays and Peristaltic Pump: ~$15.00
- Custom PCB and Waterproof Housing: ~$10.00
- **Total Estimated Manufacturing Cost:** ~$55.00 per unit.

6.5 Pricing Strategy

We will employ a **value-based pricing strategy**. High-end competitors (such as the Neptune Systems Apex) retail for over $800, making them inaccessible to local SME breeders. By keeping our processing at the edge on a low-cost ESP32, we can price the AquaGuard base hardware unit at approximately **$150.00**. For a commercial breeder, this price is justified as the system pays for itself by preventing the loss of just a single batch of export-quality fish.

6.6 Competitive Advantage

Existing premium competitors rely almost entirely on cloud-based processing. Their major weakness is that they become "dumb" hardware if the internet connection is lost. Our distinct competitive advantage is **Offline Fault Tolerance**. By shrinking our machine learning algorithm down into a TinyML model deployed directly on the microchip, our system can catch broken sensors, estimate missing data, and safely lock actuators in milliseconds without ever needing to communicate with a cloud server.

6.7 Intellectual Property Considerations

While the physical hardware utilizes off-the-shelf components (ESP32, standard probes), the core intellectual property lies in the **Edge AI Firmware**. The specific TinyML multi-variate cross-validation algorithm which correlates Temperature, pH, and TDS to isolate biofouling from true ecological events will be protected as a trade secret and evaluated for software patentability. Additionally, the "AquaGuard AI" brand name and logo will be trademarked to build market recognition.

7. BUDGET AND JUSTIFICATION

The following budget outlines the estimated costs for developing, training, and physically validating the individual Edge TinyML Gatekeeper component over the project timeline.

TABLE IV. Estimated Budget Breakdown

Expense Category	Item Description	Estimated Cost (USD)
Hardware Costs	ESP32-Microcontroller	$10.00
	Sensor Suite (Analog pH, Analog TDS, Temp)	$45.00
	Actuator Setup (Relay module, 5V Peristaltic Pump, small heater)	$25.00
	Testing Rig (tank, calibration buffers, waterproof housing, wiring)	$40.00
Software Licenses	Edge Impulse Studio (TinyML training platform)	$0.00 (Educational)
	Arduino IDE & Python environments	$0.00 (Open Source)
Cloud Services	Google Colab Pro (for synthetic fault dataset generation - 2 months)	$20.00
	Cloud MQTT Broker (for integration testing - Free tier)	$0.00
Human Effort Cost	400 hours of research, development, and testing @ $10/hour	$4,000.00
Total Estimated Cost	(Theoretical cost including student labor)	$4,140.00

8. WORK BREAKDOWN STRUCTURE (WBS)

To ensure smooth project execution over the 10-month timeline, the workload is distributed equally among all four team members. Each member is assigned dedicated research and technical deliverables tailored to their specific component, while shared milestones guarantee seamless integration across the entire AquaGuard AI ecosystem.

Note: A comprehensive, hierarchical visual diagram of the project's task distribution is available in **Appendix 4**.

8.1 Task Breakdown per Member

- **Member 1 (Author – Edge TinyML & Actuator Gatekeeper):**
  - **Data Preparation:** Download baseline freshwater datasets and create Python scripts to synthetically inject sensor faults (biofouling drift and electrical spikes).
  - **Model Development:** Train multivariate classification and data imputation models in Edge Impulse, then quantize them with TensorFlow Lite for Microcontrollers to run under 250 KB.
  - **Firmware & Gatekeeper Logic:** Write ESP32 firmware to read sensors, run real-time inference, and immediately cut power to the heater relay and pH buffer pump if a fault occurs.
  - **Hardware Testing:** Assemble a physical testing tank to benchmark detection accuracy and verify that actuator lockout occurs in under 500 milliseconds.
- **Member 2 (Cloud XAI Brain):**
  - Build a fish species database and compatibility rule engine.
  - Develop the cloud logic that generates dynamic water quality target profiles.
  - Create the user-facing mobile application and clear, explainable alert notifications.
- **Member 3 (Computer Vision Fish Tracker):**
  - Set up overhead camera hardware and capture video feeds across different tank conditions.
  - Train deep learning models to identify fish species and track swimming trajectories.
  - Detect physical stress behaviors, such as surface gasping or erratic movement.
- **Member 4 (Predictive ML Forecaster):**
  - Set up cloud data pipelines to ingest clean telemetry from Member 1 and visual stress data from Member 3.
  - Train time-series forecasting models to predict water parameter trends.
  - Build an early-warning risk model to forecast ammonia spikes and disease risks before physical symptoms appear.
- **Shared Responsibilities (All Members):**
  - **Months 1–2:** Literature review, requirement gathering, and project proposal preparation.
  - **Months 5–6:** Standardizing communication protocols, MQTT topics, and JSON data formats.
  - **Months 7–8:** Connecting edge hardware with cloud services for full system integration.
  - **Months 9–10:** Final validation testing, dissertation write-up, and viva defense.

8.2 Project Timeline Alignment

TABLE V. Project Timeline Alignment

Project Phase	Primary Focus	Responsible Member	Target Timeline	Output Deliverable
Phase 1: Foundation	Literature Review & Proposal Planning	All Members	Months 1–2	Proposal report and oral defense
Phase 2: Core Development	Synthetic Data & TinyML Training	Member 1	Months 2–5	Quantized edge AI model
	Cloud Brain & Mobile UI	Member 2	Months 2–5	Cloud rule engine & mobile app
	Video Collection & Fish Detection	Member 3	Months 2–5	Trained vision tracking model
	Data Ingestion & Predictive Modeling	Member 4	Months 2–5	Risk prediction algorithms
Phase 3: Firmware & Protocols	ESP32 Gatekeeper Logic & Relays	Member 1	Months 6–7	Working offline controller
	MQTT Messaging Architecture	All Members	Months 5–6	Verified JSON data pipeline
Phase 4: Testing & Integration	Physical Tank Stress Testing	Member 1	Months 8–9	Latency & accuracy test logs
	End-to-End System Integration	All Members	Months 7–8	Unified edge-to-cloud prototype

Phase 5: Final Delivery	Final Thesis & Presentation	All Members	Months 9–10	Final dissertation & viva defense

8.3 Workload Distribution

The workload is divided equally across the 10-month timeline:

- Each member completes approximately **400 hours** of dedicated technical development on their individual component.
- The team shares approximately **150 hours per member** for joint meetings, documentation, MQTT pipeline integration, and final testing.
- This balance ensures that every student delivers an independent, clearly defined engineering deliverable while keeping the overall ecosystem tightly connected.

9. GANTT CHART

TASK	M1	M2	M3	M4	M5	M6	M7	M8	M9	M10
Literature work	■	■								
Research design	■	■								
Data preparation (Sensor dataset & fault injection)			■							
Experiments (Edge Impulse DSP & TinyML training)			■	■						
Development (ESP32 firmware & C++ coding)				■	■					
Prototype work (Hardware relay & tank assembly)					■	■				
Testing (Physical chemical/thermal stress tests)						■	■			
Integration (MQTT cloud connection with Members 2-4)							■	■		
Evaluation (Benchmarking latency & accuracy)								■	■	
Analysis (Reviewing data & system performance)									■	
Report preparation (Final thesis writing & viva)									■	■

10.REFERENCES LIST

[1] S. M. D. Lainez, U. M. Melendres, and J. E. S. Anthony, "Development of Real-Time Water Quality Assessment Using eH2O: An Internet of Things-Based Aquaculture Farm Management System," in Proc. 2026 8th Int. Conf. Softw. Eng. Comput. Sci. (CSECS), 2026, pp. 97–102, doi: 10.1109/CSECS69124.2026.11542034.

[2] J. P. Ramirez, C. Stefanini, G. De Masi, and D. Romano, "Design of magnetic coupling-based anti-biofouling mechanism for underwater optical sensors," in Proc. OCEANS 2022 - Chennai, Chennai, India, 2022, pp. 1–6, doi: 10.1109/OCEANSChennai45887.2022.9775454.

[3] K. Guralla, J. N. Kishore, E. Narasimhulu, S. Naseema, and M. B. K. Reddy, "Real-Time Sensor Drift Detection in IoT Edge Devices Using Lightweight Machine Learning," in Proc. 2026 12th Int. Conf. Commun. Signal Process. (ICCSP), Melmaruvathur, India, 2026, pp. 841–845, doi: 10.1109/ICCSP68173.2026.11539155.

[4] M. Mondal et al., "iAirGuard: A Modular IoT Architecture with Dynamic Sampling and TinyML-Based Fault Detection for Air Quality Monitoring," in Proc. 2026 18th Int. Conf. COMmunication Syst. NETworkS (COMSNETS), Bengaluru, India, 2026, pp. 1397–1402, doi: 10.1109/COMSNETS67989.2026.11418151.

[5] A. Datta et al., "PolluSenseCheck: Cost Effective TinyML-based Air Quality Monitoring System with In-Built Sensor Fault Detection," in Proc. 2025 17th Int. Conf. COMmunication Syst. NETworkS (COMSNETS), Bengaluru, India, 2025, pp. 623–631, doi: 10.1109/COMSNETS63942.2025.10885641.

[6] S. Attarha and A. Förster, "Building Resilient IoT Systems through Resource-Efficient Sensor Fault Correction," in Proc. 2025 IEEE Annu. Congr. Artif. Intell. Things (AIoT), 2025, pp. 512–519, doi: 10.1109/AIoT66900.2025.00078.

[7] Export Development Board (EDB), Sri Lanka, "Industry Capability Report: The Ornamental Fish Industry in Sri Lanka," Mar. 2025.

11.APPENDICES

Appendix 1

Industry Needs and Water Quality Management Evidence (Sri Lanka EDB)

Description:

Official documentation from the Sri Lanka Export Development Board (EDB) website detailing persistent operational constraints faced by local ornamental fish farmers. The documentation explicitly identifies the critical shortage of technical support in water quality management and disease prevention as a primary sector bottleneck.

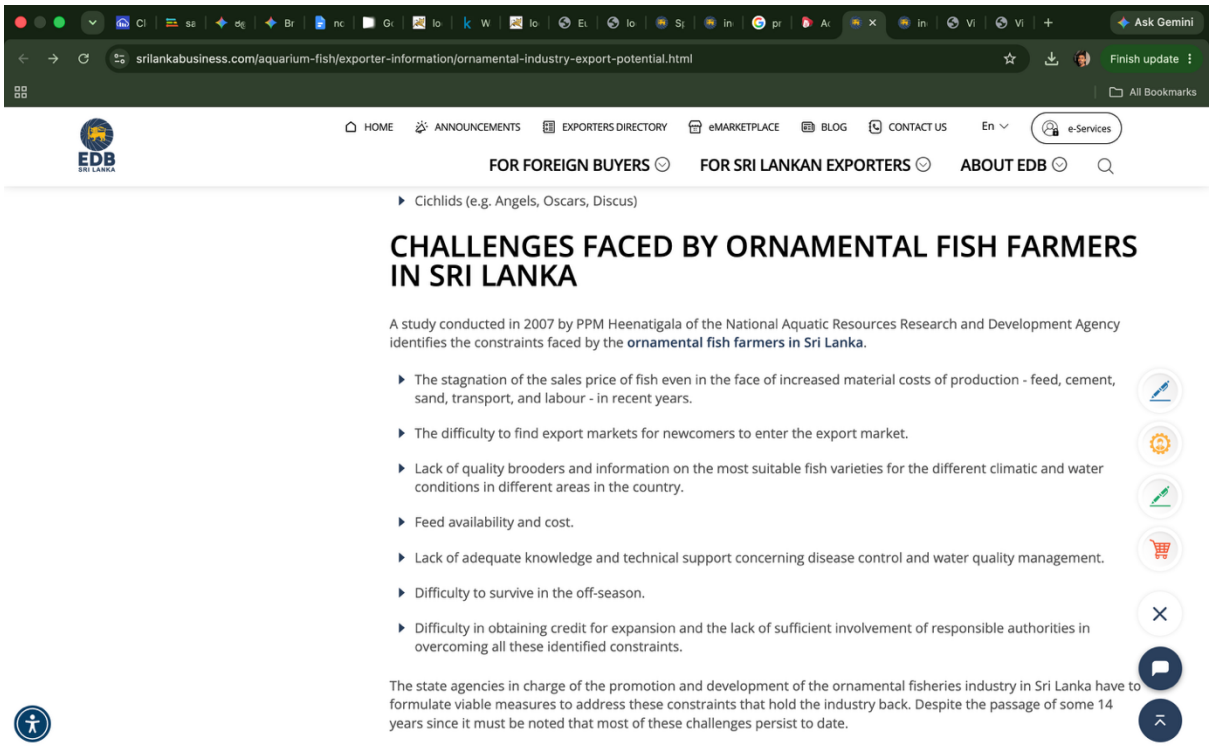


Figure A1: Official Sri Lanka EDB portal identifying water quality management and disease control deficiencies among ornamental fish farmers.

- **Source URL:** https://www.srilankabusiness.com/aquarium-fish/exporter-information/ornamental-industry-export-potential.html
- **Access Date:** August 2026
- **Key Validated Evidence:**
    1. Direct acknowledgment that lack of technical knowledge and support in *water quality management* restricts farmer success.
    2. Confirmation that fish disease outbreaks remain a primary operational hazard.
    3. Documented persistence of these structural challenges across the local aquaculture industry.

Appendix 2 — Mandatory Proposal Evidence and Decision Log

Date	Claim / Problem	Evidence Collected	Alternatives Considered	Decision Made	Supervisor Discussion
May 22, 2026	Initial component scope (just collecting IoT data and sending it to the cloud) lacked individual research depth.	Review of basic smart aquariums showed they only act as passive data pipelines without local intelligence.	1. Keep it as a basic data-gathering node. 2. Upgrade the component to process data locally at the edge.	Shift the component's focus from basic IoT data collection to edge computing by running a smart algorithm locally on the device.	Supervisor explicitly advised that instead of just working with IoT data, I should implement an algorithm directly on the Edge IoT device to increase academic rigor.
Aug 12, 2026	If running an algorithm on the edge, what specific problem should it solve to protect the fish?	Industry reports (EDB Sri Lanka) and literature showed that automated systems fail fatally when dirty sensors give false readings (the "blind trust" problem).	1. Write basic IF/ELSE rule scripts to check data. 2. Use a TinyML machine learning model for accurate fault detection.	Implement a TinyML multi-variate classification algorithm using Edge Impulse to mathematically detect sensor faults.	-
Sept 2, 2026	How to safely control the aquarium heater and pH pump if the Wi-Fi or cloud connection drops during a storm.	User observations and stakeholder feedback highlighted that local hatcheries experience frequent internet instability.	1. Rely on the Cloud Brain (Member 2) to trigger actuators. 2. Create an offline, physical safety lock	Design the Edge IoT Gatekeeper to operate 100% offline, physically locking the hardware relays itself while sending clean data to the cloud.	-

			on the edge device.		

Appendix 3 — AI Use Disclosure

AI Tool	Purpose of Use	What Was Generated/Assisted	How Output Was Verified	Final Student Contribution
Google Gemini	Brainstorming and Document Structuring	Assisted in organizing the proposal structure (Methodology, WBS, Commercialization) to align with the mandatory university template.	Cross-checked the suggested layouts and headings against the official university rubric and proposal guidelines.	I independently defined the core technical architecture, the project scope, and the decision to use TinyML on an ESP32.
Google Gemini	Grammar Polishing and Academic Tone	Refined my raw bullet points into formal, IEEE-compliant English for the Research Gap, Objectives, and Budget justification sections.	Manually reviewed all generated text to ensure it accurately reflected my original ideas. Verified all citations against the actual IEEE papers.	I provided all the foundational data, selected the 6 core research papers, calculated the actual hardware prices, and drew the system diagrams.

Appendix 4: Hierarchical Work Breakdown Structure (WBS) Diagram

Description:

A complete hierarchical breakdown of the AquaGuard AI project. This visual diagram details the specific tasks, sub-tasks, and shared integration milestones distributed across all four team members to ensure balanced workload alignment over the development timeline.

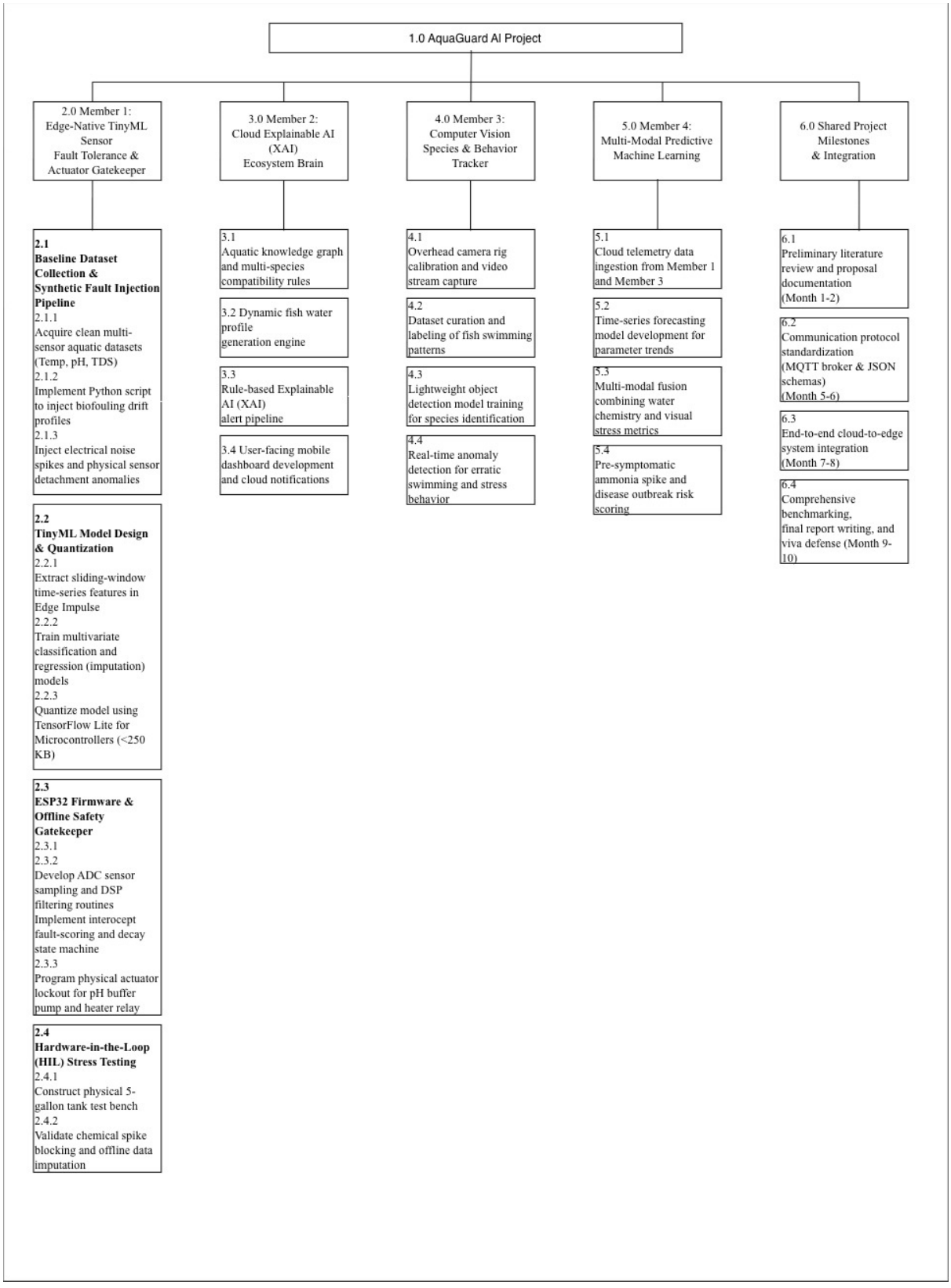


Figure A2: Visual Work Breakdown Structure outlining individual components and shared project milestones.

Appendix 5: Hardware-in-the-Loop (HIL) Physical Prototype Concept

Description:

A visual representation of the planned physical testing environment for the AquaGuard AI system. This setup highlights the Edge IoT Gatekeeper components (ESP32 microcontroller, submerged pH, TDS, and Temperature probes, and the mechanical pH buffer dosing pump) working alongside the computer vision camera module for integrated system testing.



Figure A3: Conceptual physical prototype of the AquaGuard AI hardware testing rig and sensor placement. *(Note: Image represents the planned physical architecture).*

Appendix 6: Mobile Application User Interface (UI) Concepts

Description:

The following wireframes demonstrate the end-user mobile application, visualizing the integrated outputs from all four project components: edge hardware telemetry, computer vision tracking, dynamic profiling, and predictive cloud forecasting.



Figure A4(a) Figure A4(b) Figure A4(c) Figure A4(d) Figure A4(e)

Figure A4(a) - Live Dashboard: Displays real-time, clean telemetry (Temperature, pH, TDS) ingested directly from the Edge IoT Gatekeeper.

Figure A4(b) - Sensor Diagnostics (Author's Component): Confirms the status of the on-device TinyML anomaly detection model running locally on the ESP32.

Figure A4(c) - Vision & Behavior Tracker: Visualizes the real-time computer vision output and explainable AI (XAI) insights regarding fish stress levels.

A4(d) - Eco-Profile Generator: The cloud-based rule engine calculating dynamic water parameter targets based on the identified fish species.

Figure A4(e) - Predictive AI Core: Multi-modal forecasting alerts and actionable XAI mitigation plans to prevent future biological crises.

Appendix 7: Preliminary Datasets

Description:

The following is a sample of the raw time-series dataset acquired to train the local Edge IoT Gatekeeper. This dataset is specifically utilized to train the TinyML classification model to differentiate between normal aquatic conditions and hardware-induced sensor faults.

Dataset: Supervised Fault Tolerance Dataset

- **Filename:** aquarium_fault_tolerance_dataset.csv
- **Purpose:** This dataset contains 500 hours of continuous telemetry with pre-classified binary fault labels (0 = Normal, 1 = Fault) for Temperature, pH, and Total Dissolved Solids (TDS). By comparing "true" ecological values against "measured" values, it allows the ESP32 machine learning model to accurately isolate hardware drift, electrical noise, and probe detachment.

Sample Data:

timestamp_hourly_idx	temp_true	temp_measured	temp_fault_label	ph_true	ph_measured	ph_fault_label	tds_true	tds_measured	tds_fault_label
0	25.07	25.07	0	7.27	7.27	0	384.19	384.19	0
1	25.28	25.28	0	7.33	7.33	0	382.87	382.87	0
2	25.69	25.69	0	7.06	7.06	0	380.37	380.37	0